#1 Method `dfs` takes a graph, a starting point vertex, and a few accessory arguments to keep track of the time of discovery for this vertex, and returns the same arguments updated.
#2 Increment the “time” counter. This is needed to keep track of the order of discovery of vertices.
#3 Current vertex has just been visited, so record it.
#4 Iterate over all the outgoing edges of vertex v.
#5 If a vertex in-time is still `null`, then it hasn’t been discovered yet (provided proper initialization of this argument). If the edge’s destination vertex u hadn’t already been discovered, then we should traverse the edge e and visit u.
#6 Recursively call `dfs` on the vertex u, and update all the auxiliary data, including the `time` counter.
#7 Once all outgoing edges has been traversed, increment the time by one extra unit (notice that `time` might already have been incremented in the recursive calls).
#8 Since we are leaving this vertex for good (we traversed all its outgoing edges), we can set its out-time.
#9 Return updated values for the time.

Figure 14.11 shows an example of DFS traversal, on the same graph we used to illustrate BFS, and using the same vertex, v_1, as starting point: you can see how the sequence of vertices visited is completely different (and not just because of the way we break ties about which edge to traverse first). The most noticeable detail is, perhaps, that we use a stack instead of a queue to keep track of the next vertices to be visited.